| EXP 7 | Implementation of three input nonlinear function |
|---|---|

**AIM** **To understand the implementation of a three-input nonlinear function using a Genetic Algorithm.**

**Algorithm**

1. Define the fitness function.
2. Initialize the population with random values.
3. Apply genetic operations (selection, crossover, mutation).
4. Execute the genetic algorithm for a fixed number of generations.
5. Print the final best solution found by the genetic algorithm.

**PROGRAM**

```
import random

# Define the fitness function (three-input nonlinear function)
def fitness_function(x, y, z):
    return -(x**2 + y**2 + z**2) + 10 * (math.cos(2*math.pi*x) + math.cos(2*math.pi*y) +
math.cos(2*math.pi*z))

# Initialize the population
def initialize_population(pop_size, lower_bound, upper_bound):
    return [(random.uniform(lower_bound, upper_bound), random.uniform(lower_bound,
upper_bound), random.uniform(lower_bound, upper_bound)) for _ in range(pop_size)]

# Select parents based on their fitness
def select_parents(population):
    total_fitness = sum(fitness_function(x, y, z) for x, y, z in population)
    roulette_wheel = [fitness_function(x, y, z) / total_fitness for x, y, z in population]
    parent1 = random.choices(population, weights=roulette_wheel)[0]
    parent2 = random.choices(population, weights=roulette_wheel)[0]
    return parent1, parent2

# Perform crossover to create a new generation
def crossover(parent1, parent2, crossover_prob=0.7):
    if random.random() < crossover_prob:
        crossover_point1 = random.uniform(0, 1)
        crossover_point2 = random.uniform(0, 1)
        child1 = (crossover_point1 * parent1[0] + (1 - crossover_point1) * parent2[0],
                  crossover_point1 * parent1[1] + (1 - crossover_point1) * parent2[1],
                  crossover_point1 * parent1[2] + (1 - crossover_point1) * parent2[2])

        child2 = (crossover_point2 * parent1[0] + (1 - crossover_point2) * parent2[0],
                  crossover_point2 * parent1[1] + (1 - crossover_point2) * parent2[1],
                  crossover_point2 * parent1[2] + (1 - crossover_point2) * parent2[2])
```

```
        return child1, child2
    else:
        return parent1, parent2

# Perform mutation in the population
def mutate(individual, mutation_prob=0.01):
    x, y, z = individual
    if random.random() < mutation_prob:
        x += random.uniform(-0.1, 0.1)
    if random.random() < mutation_prob:
        y += random.uniform(-0.1, 0.1)
    if random.random() < mutation_prob:
        z += random.uniform(-0.1, 0.1)
    return x, y, z

# Genetic Algorithm
def genetic_algorithm(generations, pop_size, lower_bound, upper_bound):
    population = initialize_population(pop_size, lower_bound, upper_bound)

    for gen in range(generations):
        new_population = []

        while len(new_population) < pop_size:
            parent1, parent2 = select_parents(population)
            child1, child2 = crossover(parent1, parent2)
            child1 = mutate(child1)
            child2 = mutate(child2)
            new_population.extend([child1, child2])

        population = new_population

        best_individual = max(population, key=lambda ind: fitness_function(*ind))
        print(f"Generation {gen+1}: Best individual - {best_individual}, Fitness -
{fitness_function(*best_individual)}")

    return max(population, key=lambda ind: fitness_function(*ind))

if __name___== "__main__":
    import math

    generations = 20
    pop_size = 100
    lower_bound = -1
    upper_bound = 1

    best_solution = genetic_algorithm(generations, pop_size, lower_bound, upper_bound)
    print(f"Best solution found: {best_solution}, Fitness: {fitness_function(*best_solution)}")
```

**Output:**

Generation 1: Best individual - (0.12668693082957627, -0.07898360666672744, -0.09681577754995174), Fitness - 23.963857487736647
Generation 2: Best individual - (-0.04502425092802292 4, -0.09149226175753922, -0.07642110889761108), Fitness - 26.848115698653913
Generation 3: Best individual - (0.09881252717830102, -0.018320980176099372, -0.006486844734282238), Fitness - 28.049169911738502
Generation 4: Best individual - (0.10076810073298959, 0.024980240014169874, -0.0735695903357067), Fitness - 26.873102830498375
Generation 5: Best individual - (0.060597432767771464, -0.014288110971265644, -0.11219467016230772), Fitness - 26.84365134515204
Generation 6: Best individual - (0.10118311095963373, -0.0468329060285978, -0.13083944777828332), Fitness - 24.393800322371614
Generation 7: Best individual - (0.13200663834572657, 0.07623548532952856, -0.017106370246850633), Fitness - 25.546338218130877
Generation 8: Best individual - (0.023078254489417083, -0.05476535434162602, -0.0701494698227379), Fitness - 28.344658502516364
Generation 9: Best individual - (0.2370647997882281, 0.030548132494634783, -0.3771319907309153), Fitness - 3.263695095416392
Generation 10: Best individual - (0.1708914583394496, -0.06143565874959578, -0.4312102609184485), Fitness - 4.733251445174121
Generation 11: Best individual - (0.11502207171190051, -0.11531594650855989, -0.4080366040604845), Fitness - 6.4186028551296275
Generation 12: Best individual - (0.06477535332694237, -0.19635278619993796, -0.4173092464537592), Fitness - 3.593159693639323
Generation 13: Best individual - (0.07937534865881143, -0.11982876951480932, -0.49906649396789915), Fitness - 5.809399306091991
Generation 14: Best individual - (0.33708627520981904, 0.05443962904441646, -0.46611799025371126), Fitness - -5.890234623594891
Generation 15: Best individual - (-0.013612237035674435, -0.236919098902418, -0.4558029649004369), Fitness - 0.9034566745017869
Generation 16: Best individual - (-0.2270140089312429, -0.4385262742565512, -0.4726529889030239), Fitness - -18.144025650302847
Generation 17: Best individual - (-0.23520104067610742, -0.444920412782038, -0.4699482415669208), Fitness - -18.774993590650336
Generation 18: Best individual - (-0.23520104067610742, -0.444920412782038, -0.4699482415669208), Fitness - -18.774993590650336
Generation 19: Best individual - (-0.2541562861448274, -0.4629019276106699, -0.46744410600828795), Fitness - -20.279573998864425
Generation 20: Best individual - (-0.25778206226308176, -0.5374720930981939, -0.44415p39296664798), Fitness - -20.1561429854343
Best solution found: (-0.25778206226308176, -0.5374720930981939, -0.4441539296664798), Fitness: -20.1561429854343

**Result:** Thus the above program genetic algorithm for three input non-linear function optimization executed successfully.